3.1. Sintaxis básica y conceptos fundamentales de CSS

LENGUAJES DE PROGRAMACIÓN NIVEL 3. UNIDAD 3

CSS - ESTILOS PARA LA PRESENTACIÓN VISUAL DE LA PÁGINA WEB

Financiado por la Unión Europea

Plan de Recuperación, Transformación y Resiliencia

Contenido

Introducción 3
Conceptos fundamentales: sintaxis y estructura de CSS 4
División de tareas: HTML y CSS 4
Reglas de CSS: 5
Comentarios en CSS: 6
Cómo funcionan las reglas CSS en conjunto 7
La cascada y la herencia 8
Cómo funciona la cascada 8
Ejemplo de cascada 9
Principios de herencia 11
Ejemplo de herencia 12
Excepciones y control de la herencia 13
Conflictos y solución de problemas en CSS 14
Especificidad y conflictos 14
Herramientas de desarrollo 15
Conclusión 17
Recursos adicionales 18
Enlaces a documentación oficial y tutoriales 18
Herramientas en línea para practicar CSS 18

Introducción

Te doy la bienvenida al estiloso mundo de las Hojas de Estilo en Cascada (CSS), el lenguaje que da vida y estilo a las páginas web construidas con HTML. Mientras que HTML se encarga de la estructura y el contenido, CSS se ocupa de la presentación visual, permitiéndonos transformar páginas web básicas en experiencias ricas y envolventes.

Este módulo, "Sintaxis básica y conceptos fundamentales de CSS", marca el inicio de nuestro viaje a través del capítulo dedicado a CSS, donde exploraremos cómo utilizar este poderoso lenguaje para diseñar y estilizar páginas web de manera efectiva.

Al final de este módulo, aspiramos a que comprendas la sintaxis esencial de CSS, cómo se estructuran las reglas de estilo y la importancia de la cascada y la herencia en la aplicación de estilos. Nos adentraremos también en el uso de selectores para aplicar estilos específicos a elementos HTML, y discutiremos las propiedades y valores que puedes utilizar para manipular la apariencia de una página web. Además, abordaremos el modelo de caja de CSS, un concepto crucial que subyace a la disposición de elementos en la web.

No te asustes. Iremos poco a poco para que este módulo siente las bases para que puedas empezar a experimentar con CSS y a aplicarlo a tus propios proyectos web, abriendo un mundo de posibilidades creativas para la presentación visual de tus sitios web.

Prepárate para sumergirte en la sintaxis y los conceptos que hacen de CSS una herramienta indispensable en el diseño web moderno.

Empecemos.

Conceptos fundamentales: sintaxis y estructura de CSS

CSS (Cascading Style Sheets) juega un papel crucial en el diseño web, permitiendo a los desarrolladores controlar la presentación visual de sus páginas web. Comprender la sintaxis y la estructura de CSS es esencial para aplicar estilos eficazmente y mantener tus hojas de estilo organizadas y fáciles de leer.

División de tareas: HTML y CSS

En el desarrollo web, HTML y CSS trabajan mano a mano para construir páginas web, pero cada uno tiene un rol específico que es fundamental entender para lograr diseños web efectivos y mantenibles.

- HTML (Hypertext Markup Language) es el esqueleto de la página web. Se encarga de estructurar y organizar el contenido, definiendo elementos como textos, enlaces, imágenes y otros componentes que componen la página. HTML utiliza etiquetas para marcar diferentes tipos de contenido, estableciendo la base sobre la cual se aplicarán los estilos.
- CSS (Cascading Style Sheets), por otro lado, es como el traje a medida que se adapta al esqueleto proporcionado por HTML. CSS se encarga del aspecto visual y la presentación de la página, permitiendo a los desarrolladores controlar el layout, los colores, las fuentes, los

espacios, y mucho más. Con CSS, puedes transformar el contenido estructurado por HTML en una experiencia visualmente atractiva y coherente.

Reglas de CSS:

CSS se presenta y aplica a través de reglas de estilo. Cada regla de CSS está diseñada para controlar la presentación de uno o más elementos HTML en la página web

Una regla de CSS se compone de dos partes principales: un selector y un bloque de declaración.

- Selector: Especifica el elemento HTML al que se aplicarán los estilos. Puede ser el nombre de una etiqueta HTML, una clase, un ID u otros selectores más avanzados.
- Bloque de declaración: Contiene una o más declaraciones separadas por punto y coma (;). Cada declaración incluye una propiedad CSS y un valor separado por dos puntos (:). Este bloque se encierra entre llaves ({}).

```
selector {
  propiedad: valor;
  propiedad2: valor2;
}
```

[1]

[1] Accesibilidad;
selector {
propiedad: valor;
propiedad2: valor2;
}

Un ejemplo:

```
p {
  color: blue;
  font-size: 16px;
}
```

[2]

Este ejemplo selecciona todos los elementos <p> (párrafos) y les aplica un color de texto azul y un tamaño de fuente de 16px.

Comentarios en CSS:

Los comentarios son fragmentos de texto que se incluyen en el código para explicaciones o anotaciones, y son ignorados por el navegador. En CSS, los comentarios se encierran entre /* y */.

```
/* Este es un comentario en CSS */
p {
  color: red; /* Esto hará que el texto de los párrafos sea rojo */
}
```

[3]

Usar comentarios ayuda a:

- Explicar el código: Aclarar la función de reglas o propiedades específicas, especialmente en casos complejos o no obvios.
- Organizar el código: Marcar secciones de la hoja de estilo, facilitando la navegación y la comprensión de la estructura.

---

[2] Accesibilidad:
p {
  color: blue;
  font-size: 16px;
}

[3] Accesibilidad:
/* Este es un comentario en CSS */
p {
  color: red; /* Esto hará que el texto de los parrados sea rojo */
}

- Depurar: Temporalmente desactivar partes del código sin eliminarlas, útil durante la solución de problemas.

Cómo funcionan las reglas CSS en conjunto

Las reglas CSS en una hoja de estilo se aplican a la página HTML en el orden en que aparecen, lo que es importante en el contexto de la cascada de estilos que veremos posteriormente. Si hay reglas contradictorias para el mismo elemento, la última regla leída por el navegador es la que prevalece, a menos que se especifique lo contrario con la propiedad !important o haya diferencias de especificidad entre los selectores.

El uso de reglas CSS permite a los diseñadores y desarrolladores web separar el contenido (HTML) de la presentación (CSS), lo que facilita la mantenibilidad y reutilización del código, además de permitir una mayor flexibilidad y control sobre el diseño de la página web.

La cascada y la herencia

Después de haber explorado los selectores, propiedades y valores en CSS, es crucial entender cómo estos elementos interactúan en un documento web complejo.

La cascada y la herencia son principios fundamentales que determinan cómo se aplican los estilos a los elementos HTML, resolviendo conflictos cuando múltiples reglas podrían aplicarse al mismo elemento y estableciendo cómo se transmiten los estilos de un elemento a sus descendientes.

Cómo funciona la cascada

La "cascada" en CSS describe el proceso utilizado para determinar qué reglas de estilo se aplicarán a un elemento cuando múltiples reglas tienen selectores que coinciden con el mismo elemento. La cascada sigue estas reglas básicas para resolver conflictos:

- Importancia: Las reglas marcadas con !important tienen la máxima prioridad. Sin embargo, el uso excesivo de !important se considera una mala práctica porque rompe el flujo natural de la cascada.
- Especificidad: Cuando dos reglas tienen diferentes selectores, pero podrían aplicarse al mismo elemento, la regla con el selector de mayor especificidad gana. La especificidad se calcula en función de la cantidad y el tipo de componentes del selector (por ejemplo, los ID son más específicos que las clases, y las clases son más específicas que los selectores de tipo).

- Orden de origen: Si dos reglas tienen la misma importancia y especificidad, la última regla en el código CSS será la que se aplique.

Ejemplo de cascada

Imagina que tenemos un documento HTML con varios elementos <p> (párrafos), y queremos aplicar diferentes estilos de color a estos párrafos usando CSS. Podríamos tener reglas CSS en nuestra hoja de estilo que se vean algo así:

```
/* Regla 1: Selector de tipo que aplica a todos los elementos <p> */
p {
  color: red; /* Establece el color de texto de todos los párrafos a rojo */
}

/* Regla 2: Selector de clase que aplica a elementos <p> con la clase 'destacado' */
.destacado {
  color: blue; /* Establece el color de texto de los párrafos destacados a azul */
}

/* Regla 3: Selector de ID que aplica al elemento <p> con el ID 'especial' */
#especial {
  color: green; /* Establece el color de texto del párrafo especial a verde */
}
```

[4]

[4] Accesibilidad:

/* Regla 1: Selector de tipo que aplica a todos los elementos <p>*/
p {
color: red; /* Establece el color de texto de todos los párrafos a rojo*/
}

/* Regla 2: Selector de clase que aplica a los elementos <p> con la clase “destacado”*/
.destacado {
color: blue; /* Establece el color de texto de los párrafos destacados a azul*/
}

/* Regla 3: Selector de id que aplica al elemento <p> con el id “especial”*/
#especial {
color: green; /* Establece el color de texto del párrafo especial a verde*/
}

Y nuestro HTML podría verse así:

```
<p>Este párrafo será rojo.</p>
<p class="destacado">Este párrafo será azul debido a su clase.</p>
<p id="especial">Este párrafo será verde debido a su ID.</p>
<p class="destacado" id="especial">Este párrafo será verde debido a su ID, que tiene
```

[5]

Cómo funciona la cascada en este ejemplo:

- Todos los párrafos recibirán inicialmente el estilo de la Regla 1 (color: red;), ya que esta regla selecciona a todos los elementos .
- El segundo párrafo, al tener la clase destacado, también es seleccionado por la Regla 2. Aquí entra en juego la cascada: aunque la Regla 1 también se aplica, la Regla 2 tiene mayor especificidad debido a que se utiliza un selector de clase, por lo tanto, este párrafo será azul.
- El tercer párrafo, al tener el ID especial, es seleccionado por la Regla 3. Los ID tienen aún más especificidad que las clases, así que este párrafo será verde, ignorando el color rojo aplicado por la Regla 1.
- El cuarto párrafo es interesante porque cumple con las condiciones tanto de la Regla 2 como de la Regla 3. En este caso, aunque ambas reglas se aplican, la Regla 3 prevalece porque un ID tiene mayor especificidad que una clase, haciendo que este párrafo también sea verde.

[5] Accesibilidad:

Este párrafo será rojo.
Este párrafo será azul debido a su clase.
<p id="especial'>Este párrafo será verde debido a su 1D.
Este párrafo será verde debido a su ID, que tiene</P>

Principios de herencia

La herencia en CSS es el mecanismo por el cual algunos estilos aplicados a un elemento padre se transmiten a sus elementos hijos. No todas las propiedades de CSS se heredan, pero muchas relacionadas con el texto, como color y font-family, lo hacen por defecto.

Hay dos aspectos a considerar con respecto al tema de la herencia:

- Herencia explícita: Puedes usar la propiedad inherit para aplicar explícitamente a un elemento el valor de una propiedad de su elemento padre.

```
p {
  color: inherit; /* El color del texto de los párrafos será el mismo que el de su e
}
```

[6]

- Anulación de la herencia: Aunque un elemento herede estilos de su elemento padre, cualquier regla aplicada directamente al elemento hijo prevalecerá sobre la herencia.

```
/* Suponiendo que el elemento padre tenga un color de texto rojo */
p {
  color: blue; /* Este color azul anulará el color heredado del padre */
}
```

[7]

[6] Accesibilidad:

```
p {
  color: inherit /* El color del texto de los párrafos será el color del elemento padre”*/
}
```

[7] Accesibilidad

```
/* Suponiendo que el elemento padre tenga un color de texto rojo”*/
p {
  color: blue /* Este color azul anulará el color rojo heredado del padre*/
}
```

Ejemplo de herencia

Imagina que tenemos un documento HTML que incluye un elemento que, a su vez, contiene un elemento <p> (párrafo):

```
<div class="contenedor">
  Este es texto dentro del div.
  <p>Este es un párrafo dentro del div.</p>
</div>
```

[8]

Y tenemos la siguiente regla CSS aplicada al elemento :

```
.contenedor {
  color: blue; /* Establece el color de texto de los elementos dentro del div a azul
  font-size: 20px; /* Establece el tamaño de fuente de los elementos dentro del div a
}
```

[9]

Cómo Funciona la Herencia en Este Ejemplo:

- El elemento con la clase contenedor se estilizará directamente según las reglas definidas: su texto será azul y tendrá un tamaño de fuente de 20px.
- El elemento <p> (párrafo) dentro del no tiene reglas de estilo específicas aplicadas directamente a él en este ejemplo. Sin embargo, debido a la herencia en CSS, heredará algunas de las propiedades de estilo de su elemento padre . En este caso, heredará el color: blue; pero no todas las propiedades se heredan de manera predeterminada, como el margin o el padding.

---

[8] Accesibilidad.
```
<div class="contenedor">
   Este es texto dentro del div.
   <p>Este es un párrafo dentro del div.</p>
</div>
```

[9] Accesibilidad:
```
.contenedor {
  color: blue; /* Establece el color de texto de los elementos dentro del div a azul ”*/
  font-size: 20px; /* Establece el tamaño de fuente de los elementos dentro del div a 20 pixeles ”*/
}
```

- Como resultado, el texto dentro del párrafo también aparecerá en azul y con un tamaño de fuente de 20px, a pesar de que no se aplicaron estilos directamente al elemento <p>. Esto demuestra cómo la herencia puede simplificar la aplicación de estilos comunes a varios elementos, manteniendo la coherencia en todo el diseño.

Excepciones y control de la herencia

No todas las propiedades CSS se heredan de manera predeterminada. Propiedades como margin, padding, border, y background no se heredan, lo que significa que los elementos hijos no adoptarán estos estilos de sus padres a menos que se especifique explícitamente.

Puedes controlar la herencia utilizando las palabras clave inherit, initial, y unset en las propiedades CSS:

- inherit fuerza a un elemento a heredar una propiedad específica de su elemento padre.
- initial establece la propiedad a su valor predeterminado, ignorando la herencia.
- unset restablece la propiedad a su valor heredado si se hereda por defecto, o al valor inicial si no.

Conflictos y solución de problemas en CSS

A medida que las hojas de estilo se vuelven más complejas y se aplican a documentos HTML igualmente complejos, es inevitable encontrarse con conflictos entre reglas CSS que compiten por aplicar estilos a los mismos elementos. Entender cómo resolver estos conflictos y cómo utilizar las herramientas de desarrollo disponibles en los navegadores modernos es esencial para cualquier desarrollador web.

Especificidad y conflictos

La especificidad es un sistema de puntuación que los navegadores utilizan para determinar qué reglas CSS tienen prioridad sobre otras cuando más de una regla podría aplicarse a un elemento. La especificidad se calcula en función de los diferentes tipos de selectores utilizados en una regla CSS y su planteamiento jerárquico es el siguiente:

- Los selectores de ID tienen la especificidad más alta.
- Los selectores de clase, atributos y pseudo-clases vienen después.
- Los selectores de tipo y pseudo-elementos tienen la especificidad más baja.
- La declaración !important (aunque se recomienda evitar su uso cuando sea posible) sobrepasa otros niveles de especificidad.

Cuando dos reglas en conflicto tienen la misma especificidad, la regla que aparece última en el CSS es la que se aplica.

Veamos el siguiente ejemplo:

```
#id {
  color: red; /* Especificidad alta */
}

.clase {
  color: blue; /* Especificidad media, pero no se aplica debido a la regla del ID */
}

p {
  color: green; /* Especificidad baja, superada por las reglas de clase e ID */
}
```

[10]

En este ejemplo, si un elemento <p> con la clase .clase y el ID #id está presente, el color de texto aplicado sería rojo debido a la especificidad más alta del selector de ID.

Herramientas de desarrollo

Las herramientas de desarrollo del navegador son esenciales para depurar y experimentar con CSS. Permiten inspeccionar los elementos de una página, ver qué estilos se están aplicando y cómo se calcula la especificidad, y modificar los estilos en tiempo real para ver los efectos inmediatamente.

- Inspección de elementos: Puedes utilizar la herramienta de inspección para seleccionar cualquier elemento en la página y ver todos los estilos CSS que se le aplican, incluyendo aquellos heredados o anulados.

[10] Accesibilidad:

```
#id {
  color: red; /* Especificidad alta */
}

.clase {
  color: blue; /* Especificidad media, pero no se aplica debido a la regla del ID */
}

p {
  color: green; /* Especificidad baja, superada por las reglas de clase ID */
}
```

- Edición en vivo: Las herramientas de desarrollo permiten editar CSS directamente dentro del navegador, facilitando la experimentación y la solución rápida de problemas.
- Visualización del modelo de caja: Muchas herramientas de desarrollo ofrecen una representación visual del modelo de caja de un elemento, ayudando a entender cómo el margen, el borde y el relleno afectan al diseño.

Las herramientas de desarrollo más comunes se encuentran en navegadores como Chrome (Herramientas para Desarrolladores de Chrome), Firefox (Herramientas para Desarrolladores de Firefox) y Edge (Herramientas para Desarrolladores de Edge).

Conclusión

A lo largo de este módulo, hemos emprendido un viaje detallado a través de los fundamentos de CSS, explorando desde la sintaxis básica y la estructura hasta los conceptos avanzados como la cascada y la herencia y el modelo de caja.

El dominio de CSS es una habilidad esencial para cualquier desarrollador web, ya que proporciona las herramientas necesarias para crear experiencias visuales atractivas y accesibles. La capacidad de aplicar estilos eficazmente, comprender cómo se aplican esos estilos y resolver problemas cuando las cosas no van según lo planeado, son fundamentales para el desarrollo web moderno.

Al avanzar en tu camino de aprendizaje, te animamos a experimentar con los conceptos y técnicas que has aprendido. La práctica es clave para consolidar tus habilidades en CSS y para descubrir nuevas y emocionantes formas de mejorar la presentación de tus proyectos web.

Nos vemos en el siguiente módulo.

Recursos adicionales

Aquí tienes algunas recomendaciones valiosas que pueden ayudar a profundizar en el conocimiento de CSS, practicar habilidades y experimentar con estilos en tiempo real:

Enlaces a documentación oficial y tutoriales

- MDN Web Docs (Mozilla Developer Network): Ofrece una documentación exhaustiva y tutoriales sobre CSS, desde conceptos básicos hasta características avanzadas. (https://developer.mozilla.org/en-US/docs/Web/CSS W3Schools CSS Tutorial)
- W3Schools CSS Tutorial: Proporciona tutoriales sencillos y ejemplos prácticos para aprender CSS desde cero (https://www.w3schools.com/css/)
- CSS-Tricks: Un sitio web dedicado a consejos, trucos y técnicas avanzadas en CSS. Ideal para aquellos que buscan mejorar sus habilidades y resolver problemas específicos. (https://css-tricks.com/)

Herramientas en línea para practicar CSS

- CodePen: Una plataforma social de desarrollo que permite escribir código en el navegador y ver los resultados en tiempo real. Es excelente para experimentar con CSS y compartir tus creaciones. (https://codepen.io/)
- CSS Zen Garden: Un proyecto clásico que demuestra lo que se puede lograr con diseño CSS. Puedes descargar los archivos HTML y CSS y experimentar con el diseño. (https://csszengarden.com/)